

Another example of openness: the Python Software Foundation (PSF) has been working to increase diversity in the Python community. Some encouraging results are already in. The 2013–2014 PSF board saw the first women elected directors: Jessica McKellar and Lynn Root. In 2015, Diana Clarke chaired PyCon North America in Montréal, where about one-third of the speakers were women. PyLadies became a truly global movement, and I am proud that we have so many PyLadies chapters in Brazil.

If you are a Pythonista but you have not engaged with the community, I encourage you to do so. Seek the PyLadies or Python Users Group (PUG) in your area. If there isn't one, create it. Python is everywhere, so you will not be alone. Travel to events if you can. Join live events too. During the Covid-19 pandemic I learned a lot in the “hallway tracks” of online conferences. Come to a PythonBrasil conference—we've had international speakers regularly for many years now. Hanging out with fellow Pythonistas brings real benefits besides all the knowledge sharing. Like real jobs and real friendships.

I know I could not have written this book without the help of many friends I made over the years in the Python community.

My father, Jairo Ramalho, used to say “Só erra quem trabalha,” Portuguese for “Only those who work make mistakes,” great advice to avoid being paralyzed by the fear of making errors. I certainly made my share of mistakes while writing this book. The reviewers, editors, and early release readers caught many of them. Within hours of the first edition early release, a reader was reporting typos in the errata page for the book. Other readers contributed more reports, and friends contacted me directly to offer suggestions and corrections. The O'Reilly copyeditors will catch other errors during the production process, which will start as soon as I manage to stop writing. I take responsibility and apologize for any errors and suboptimal prose that remains.

I am very happy to bring this second edition to conclusion, mistakes and all, and I am very grateful to everybody who helped along the way.

I hope to see you soon at some live event. Please come say hi if you see me around!

Further Reading

I will wrap up the book with references regarding what it is to be “Pythonic”—the main question this book tried to address.

Brandon Rhodes is an awesome Python teacher, and his talk “[A Python Aesthetic: Beauty and Why I Python](#)” is beautiful, starting with the use of Unicode U+00C6 (LATIN CAPITAL LETTER AE) in the title. Another awesome teacher, Raymond Hettinger, spoke of beauty in Python at PyCon US 2013: “[Transforming Code into Beautiful, Idiomatic Python](#)”.

The “[Evolution of Style Guides](#)” thread that Ian Lee started on Python-ideas is worth reading. Lee is the maintainer of the [pep8](#) package that checks Python source code for PEP 8 compliance. To check the code in this book, I used [flake8](#), which wraps [pep8](#), [pyflakes](#), and Ned Batchelder’s [McCabe complexity plug-in](#).

Besides PEP 8, other influential style guides are the [Google Python Style Guide](#) and the [Pocoo Styleguide](#), from the team that brought us Flake, Sphinx, Jinja 2, and other great Python libraries.

The Hitchhiker’s Guide to Python! is a collective work about writing Pythonic code. Its most prolific contributor is Kenneth Reitz, a community hero thanks to his beautifully Pythonic requests package. David Goodger presented a tutorial at PyCon US 2008 titled “[Code Like a Pythonista: Idiomatic Python](#)”. If printed, the tutorial notes are 30 pages long. Goodger created both reStructuredText and docutils—the foundations of Sphinx, Python’s excellent documentation system (which, by the way, is also the [official documentation system](#) for MongoDB and many other projects).

Martijn Faassen tackles the question head-on in “[What is Pythonic?](#)” In the python-list, there is a thread with [that same title](#). Martijn’s post is from 2005, and the thread from 2003, but the Pythonic ideal hasn’t changed much—neither has the language, for that matter. A great thread with “Pythonic” in the title is “[Pythonic way to sum n-th list element?](#)”, from which I quoted extensively in the “[Soapbox](#)” on page 427.

PEP 3099 — [Things that will Not Change in Python 3000](#) explains why many things are the way they are, even after the major overhaul that was Python 3. For a long time, Python 3 was nicknamed Python 3000, but it arrived a few centuries sooner—to the dismay of some. PEP 3099 was written by Georg Brandl, compiling many opinions expressed by the *BDFL*, Guido van Rossum. The “[Python Essays](#)” page lists several texts by Guido himself.

Symbols

- `!=` (not equal to) operator, 577
- `!r` conversion field, 12
- `%` (modulo) operator, 5, 12
- `%r` placeholder, 12
- `*` (star) operator, 10, 36-37, 50-56, 240, 572-574
- `**` (double star) operator, 80, 240
- `*=` (star equals) operator, 53-56, 580-585
- `_` symbol, 42
- `_new*`, 843-845
- `+` operator, 10, 50-56, 566-572
- `+=` (addition assignment) operator, 53-56, 580-585
- `+ELLIPSIS` directive, 7
- `:=` (Walrus operator), 26
- `<` (less than) operator, 577
- `<=` (less than or equal to) operator, 577
- `==` (equality) operator, 206, 225, 577
- `>` (greater than) operator, 577
- `>=` (greater than or equal to) operator, 577
- `@` sign, 574-576
- `@asyncio.coroutine` decorator, 777
- `@cached_property`, 856
- `@contextmanager` decorator, 664-669
- `@dataclass`
 - default settings, 180
 - example using, 187-189
 - field options, 180-183
 - init-only variables, 186
 - keyword parameters accepted by, 179
 - post-init processing, 183-185
 - typed class attributes, 185
 - `__hash__` method, 180
- `@typing.overload` decorator, 520-526
- `[]` (square brackets), 6, 26, 35, 49
- `\` (backslash), 26
- `\` line continuation escape, 26
- `\N{}` (Unicode literals escape notation), 136
- `_` symbol, 41
- `__` (double underscore), 3
- `__abs__`, 11
- `__add__`, 11
- `__bool__`, 13
- `__bytes__`, 365
- `__call__`, 360
- `__class__`, 870, 908
- `__contains__`, 7, 93
- `__delattr__`, 872
- `__delete__`, 879
- `__del__`, 219
- `__dict__`, 870
- `__dir__`, 872
- `__enter__`, 658
- `__eq__`, 411-416
- `__exit__`, 658
- `__format__`, 365, 370, 418-425, 428
- `__getattr__`, 872
- `__getattr__`, 407-411, 872
- `__getitem__`, 5-8, 49, 403-407
- `__get__`, 879
- `__hash__`, 180, 411-416
- `__iadd__`, 53
- `__init_subclass__`, 914-921
- `__init__`, 9, 183, 399
- `__invert__`, 563
- `__iter__`, 603-610
- `__len__`, 5-8, 17, 403-407
- `__missing__`, 91-95

- `__mro__`, 908
- `__mul__`, 11
- `__name__`, 908
- `__neg__`, 563
- `__post_init__`, 183
- `__pos__`, 563
- `__prepare__`, 950-952
- `__repr__`, 11-13, 364, 399
- `__setattr__`, 872
- `__setitem__`, 49
- `__set__`, 879
- `__slots__`, 384-388, 411, 870, 921
- `__str__`, 13, 364
- `{}` (curly brackets), 26
- `|` (pipe) operator, 80, 686
- `|=` (pipe equals) operator, 80
- `...` (ellipsis), 7, 49

A

- abc.ABC class, 449
- abc.Iterable, 281
- abc.Sequence, 281
- ABCs (abstract base classes)
 - ABC syntax details, 457
 - defining and using ABCs, 451-457
 - further reading on, 482
 - goose typing and, 442-445
 - overview of, 481
 - in Python standard library, 449-451
 - Soapbox discussion, 484-486
 - structural typing with, 464-466
 - subclassing ABCs, 447-449, 458-460
 - type hints (type annotations), 278-280
 - UML class diagrams, 14-15
 - usage of register, 463
 - virtual subclasses of ABCs, 460-463
- abs built-in function, 10
- actual type parameters, 544
- addition assignment (`+=`) operator, 53-56, 580-585
- aliasing, 204-208
- anonymous functions, 236, 251, 311
- Any type, 266-269
- AnyStr, 286
- `.append` method, 67
- apply function, 234-236
- arguments
 - freezing with `functools.partial`, 247
 - key argument, 74
 - keyword-only arguments, 242
- arrays, 23, 59-62
- as keyword, 41
- assignment expression (`:=`), 26
- asynchronous generators, 238, 777
- asynchronous programming
 - asynchronous context managers, 786-787
 - asyncio script example, 778-781
 - avoiding CPU-bound traps, 826
 - awaitables, 781
 - benefits of, 825
 - Curio project, 821-824
 - delegating tasks to executors, 797-798
 - enhancing asyncio downloader, 787-797
 - further reading on, 828
 - iteration and iterables, 811-821
 - myth of I/O-bound systems, 826
 - overview of, 827
 - relevant terminology, 777
 - significant changes to, 776
 - Soapbox discussion, 829
 - topics covered, 775
 - type hinting asynchronous objects, 824
 - writing asyncio servers, 799-811
- Asynchronous Server Gateway Interface (ASGI), 732
- asyncio package
 - achieving peak performance with, 785
 - documentation, 776
 - downloading with, 782-786
 - enhancing asyncio downloader, 787-797
 - example script, 778-781
 - queue implementation by, 69
 - writing asyncio servers, 799-811
- asyncpg, 786
- attribute descriptors (see also attributes; dynamic attributes and properties)
 - attribute validation, 880-891
 - descriptor docstring and overriding deletion, 902
 - descriptor usage tips, 900-902
 - further reading on, 904
 - methods as descriptors, 898-900
 - overriding versus nonoverriding, 855, 892-898
 - overview of, 903
 - purpose of, 879
 - relevant terminology, 881
 - significant changes to, 880

- Soapbox discussion, 905
 - topics covered, 879
- attributes (see also attribute descriptors;
 - dynamic attributes and properties)
 - dynamic attribute access, 407-411
 - handling attribute deletion, 868
 - overriding class attributes, 389-391
 - private and protected, 382-384
 - properties and up-front costs, 394
 - safety versus security in private, 395
 - using attribute descriptors for validation, 880-891
 - using properties for attribute validation, 857-860
 - virtual attributes, 835
- augmented assignment operators, 53-56, 580-585
- averages, computing, 643-645
- await keyword, 781

B

- backslash (\), 26
- behavioral subtyping, 268
- binary records, parsing with struct, 118
- binary sequences, 120 (see also Unicode text
 - versus bytes)
- bisect module, 58
- blue tool, 259
- BOMs (byte-order marks), 129
- bool type, 13
- Boolean values, custom types and, 13
- built-in functions, 238
- byte sequences, 128 (see also Unicode text
 - versus bytes)
- bytecode, disassembling, 310

C

- call by sharing, 213
- callable objects
 - nine types of, 237-239
 - user-defined, 239
 - using iter() with, 598-599
- Callable type, 291
- card deck example, 5-8
- Cartesian products, 27-29
- case folding, 142
- chain generator, 633
- ChainMap, 95
- characters

- finding Unicode by name, 151-153
 - numeric meaning of, 153-155
- Chardet library, 129
- cladistics, 444
- class metaprogramming
 - benefits and drawbacks of, 907
 - built-in class factory, 909
 - class factory function, 911-914
 - classes as objects, 908
 - enhancing classes with class decorators, 922-925
 - further reading on, 954
 - import time versus runtime, 925-931
 - __init_subclass__, 914-921
 - metaclass basics, 931-942
 - metaclass issues, 947-949
 - metaclass solution for checkedlib.py, 942-947
 - overview of, 953
 - __prepare__ method, 950-952
 - significant changes to, 908
 - Soapbox discussion, 956
 - useful applications of metaclasses, 952
- classes (see also protocols)
 - as callable objects, 238
 - implementing generic classes, 541-544
 - topics covered, xxi
 - undocumented classes, 557
- classic refactoring strategy, 342-346
- classmethod decorator, 369
- client codes, 632
- clock decorators
 - class-based, 335
 - parameterized, 332-334
- closures (see decorators and closures)
- cls.mro(), 909
- cls.__bases__, 908
- cls.__qualname__, 909
- cls.__subclasses__(), 909
- code examples, obtaining and using, xxiv
- code points, 119, 161
- code smells, 163, 190-192, 446
- codecs, 123
- Collection API, 14-15
- collections.abc module
 - abstract base classes defined in, 449
 - ChainMap, 95
 - Counter, 96
 - defaultdict and OrderedDict, 85, 95

- Mapping and MutableMapping ABCs, 83
 - multiple inheritance in, 502
 - UserDict, 97
 - collections.deque class, 67-70
 - collections.namedtuple, 5, 169-172
 - Command pattern, 355-357
 - comments and questions, xxv
 - comparison operators, 206, 577-580
 - computed properties
 - caching properties with functools, 855-857
 - data-driven attribute creation, 846-848
 - properties that compute values, 845
 - property caching, 853-855
 - property overriding existing attributes, 852
 - property to retrieve linked records, 848-852
 - concatenation, 50-56
 - concurrency models
 - basics of concurrency, 696
 - benefits of concurrency, 695
 - further reading on, 734-739
 - Global Interpreter Lock impact, 713-715
 - Hello World example, 701-712
 - indefinite loops and sentinels, 721
 - multicore processors and, 725-733
 - overview of, 733
 - process pools, 716-725
 - Python programming concepts, 699-701
 - relevant terminology, 697-699
 - significant changes to, 696
 - Soapbox discussion, 739-741
 - structured concurrency, 823
 - topics covered, 696
 - concurrent executors
 - concurrent web downloads, 744-754
 - downloads with progress display and error handling, 762-772
 - Executor.map, 758-761
 - further reading on, 772
 - launching processes with concurrent.futures, 754-758
 - overview of, 772
 - purpose of, 743
 - significant changes to, 743
 - Soapbox discussion, 773
 - concurrent.futures
 - downloading with, 749-750
 - launching processes with, 754-758
 - consistent-with relationships, 268
 - Container interface, 15
 - container sequences, 22, 73
 - contention, 699
 - context managers
 - @contextmanager decorator, 664-669
 - asynchronous, 786-787
 - asynchronous generators as, 817
 - contextlib utilities, 663
 - creative uses for, 658
 - demonstrations of, 659-662
 - methods included in interface, 658
 - parenthesized in Python 3.10, 663
 - purpose of, 658
 - contravariance (see variance)
 - control flow, xxi (see also asynchronous programming; concurrent executors; concurrency models; iterators; generators; with, match, and else blocks)
 - cooperative multiple inheritance, 497
 - copies
 - deep, 211-213
 - shallow, 208-211
 - coroutine objects, 642
 - coroutines
 - computing running averages, 643-645
 - definition of term, 698
 - further reading on, 652
 - generator-based, 777
 - generic type hints for, 650
 - Global Interpreter Lock impact, 715
 - overview of, 652
 - returning values from, 646-650
 - significant changes to, 594
 - Soapbox discussion, 654
 - spinners (loading indicators) using, 706-710
 - topics covered, 594
 - types of, 777
 - understanding classic, 641
 - Counter, 96
 - covariance (see variance)
 - CPU-bound systems, 826
 - Curio project, 821-824
 - curly brackets ({}), 26
- ## D
- Dask, 727
 - data attributes (see attributes)
 - data class builders
 - @dataclass, 179-189
 - classic named tuples, 169-172

- data class as code smell, 190-192
 - further reading on, 196
 - main features, 167-169
 - overview of, 164-167, 195
 - pattern matching class instances, 192-195
 - significant changes to, 164
 - Soapbox discussion, 197
 - topics covered, 163
 - type hints, 173-179
 - typed named tuples, 172
- data descriptors (see overriding descriptors)
- data model (see Python Data Model)
- data science, 727
- data structures, xxi (see also data class builders;
 - dictionaries and sets; object references;
 - sequences; Unicode text versus bytes)
- data wrangling
 - with dynamic attributes, 836-838
 - flexible object creation, 843-845
 - invalid attribute name problem, 842
 - JSON-like data, 838-842
- decimal.Decimal class, 565
- decoding (see also Unicode text versus bytes)
 - basics of, 123-124
 - definition of, 119
 - understanding encode/decode problems, 125-131
- decorator-enhanced strategy pattern, 353-355
- decorators and closures
 - classmethod versus staticmethod, 369
 - closure basics, 311-314
 - closures in lis.py, 685
 - decorator basics, 304-306
 - decorator execution, 306
 - decorator implementation, 317-320
 - decorators in Python standard library, 320-323
 - enhancing classes with class decorators, 922-925
 - further reading on, 336
 - nonlocal declarations, 315-317
 - overview of, 336
 - parameterized decorators, 329-335
 - purpose of, 303
 - registration decorators, 308
 - significant changes to, 304
 - Soapbox discussion, 338-340
 - topics covered, 303
 - variable scope rules, 308-310
- deep copies, 211-213
- defaultdict, 85, 90
- defensive programming, 87, 440-442
- del statement, 219-221, 868
- delegating generators, 632
- deprecated collection types, 272
- deque (double-ended queue), 59, 67-70
- descriptor classes, 881
- descriptor instances, 881
- descriptors, 177, 879 (see also attribute descriptors)
- design patterns (see functions, design patterns with first-class)
- destructuring, 40
- diacritics, normalization and, 144-148
- dictcomps (dict comprehensions), 79
- dictionaries and sets
 - automatic handling of missing keys, 90-95
 - consequences of how dict works, 102
 - consequences of how set works, 107
 - dictionary views, 101-102
 - further reading on, 113
 - immutable mappings, 99
 - internals of, 78
 - modern dict syntax, 78-81
 - overview of, 112
 - pattern matching with mappings, 81-83
 - set operations, 107-110
 - set operations on dict views, 110
 - set theory, 103-106
 - significant changes to, 78
 - Soapbox discussion, 114
 - standard API of mapping types, 83-90
 - topics covered, 77
 - variations of dict, 95-99
- dir([object]) function, 870
- dis module, 310
- displays, formatting, 370-374
- distributed task queues, 732
- Django generic views mixins, 504-507
- doctest package
 - documentation, xxii
 - ellipsis in, 7
- double star (**) operator, 80, 240
- double underscore (__), 3
- double() function, 466-468
- Dublin Core Schema, 187
- duck typing, 87, 261, 302, 402, 427, 431, 470
- dunder methods, 4

- dynamic attributes and properties
 - coding property factories, 865-868
 - computed properties, 845-857
 - data wrangling with dynamic attributes, 836-845
 - dynamic versus virtual attributes, 835
 - essential attributes and functions for
 - attribute handling, 869-873
 - further reading on, 873
 - handling attribute deletion, 868
 - overview of, 873
 - property class, 860-865
 - significant changes to, 836
 - Soapbox discussion, 875-877
 - using properties for attribute validation, 857-860
- dynamic protocols, 403, 435
- dynamic type, 266-269

E

- ellipsis (...), 7, 49
- else blocks, 687-689
- emojis
 - building, 118
 - console font and, 136
 - finding characters by name, 151-153
 - in the Museum of Modern Art, 160
 - increasing issues with, 162
 - UCS-2 versus UTF-16 encoding, 124
 - varied support for, 152
- encoding (see also Unicode text versus bytes)
 - basics of, 123-124
 - definition of, 119
 - encoding defaults, 134-139
 - understanding encode/decode problems, 125-131
- equality (==) operator, 206, 225, 577
- error handling, in network I/O, 762-772
- execution units, 697
- Executor.map, 758-761
- executors, delegating tasks to, 797-798 (see also concurrent executors)
- explicit self argument, 905

F

- f-string syntax
 - benefits of, 5
 - delegation of formatting by, 370

- string representation using special methods, 12
- fail-fast philosophy, 87, 417, 440-442
- FastAPI framework, 800-804
- FIFO (first in, first out), 59, 67
- filter function, 27, 234-236
- filtering generator functions, 619
- first-class functions (see functions, as first-class objects)
- first-class objects, 231
- flake8 tool, 259
- flat sequences, 22, 73
- flawed typing, 296
- fluentpython.com, xxiii
- ForkingMixIn, 503
- formal type parameters, 544
- format() function, 370
- forward reference problem, 538
- free variables, 313, 685
- frozenset, 103 (see also dictionaries and sets)
- function decorators (see decorators and closures)
- function parameters, introspection of, 232
- function-class duality, 876
- function-oriented refactoring strategy, 347-350
- functional programming
 - packages for, 243-249
 - with Python, 250
- functions
 - abs built-in function, 10
 - dir([object]) function, 870
 - disassembling bytecode of, 310
 - double() function, 466
 - filter, map, and reduce functions, 27, 234
 - format() function, 370
 - getattr function, 871
 - getattr(object, name[, default]) function, 871
 - globals() function, 351
 - hasattr function, 871
 - higher-order functions, 234-236
 - id() function, 206
 - iter() function, 596
 - len() function, 6
 - map function, 27
 - max() function, 521
 - repr() function, 364
 - setattr function, 871

- setattr(object, name, value) function, 871, 871
- single dispatch generic functions, 324-329
- str() function, 13, 364
- super() function, 411, 488
- zip() function, 416
- functions, as first-class objects (see also decorators and closures)
 - anonymous functions, 236
 - callable objects, 237-239
 - definition of term, 231
 - flexible parameter handling and, 240-243
 - further reading on, 250
 - higher-order functions, 234-236
 - overview of, 249
 - packages for functional programming, 243-249
 - significant changes to, 232
 - Soapbox discussion, 250
 - topics covered, xxi
 - treating functions like objects, 232-234
 - user-defined callable types, 239
- functions, design patterns with first-class
 - Command pattern, 355-357
 - decorator-enhanced strategy pattern, 353-355
 - dynamic languages and, 341
 - further reading on, 358
 - overview of, 357
 - refactoring strategies, 342-353
 - significant changes to, 342
 - Soapbox discussion, 359
- functions, type hints in
 - annotating positional only and variadic parameters, 295
 - benefits and drawbacks of, 253
 - flawed typing and strong testing, 296
 - further reading on, 298
 - gradual typing, 254-260
 - overview of, 297
 - significant changes to, 254
 - Soapbox discussion, 299-302
 - supported operations and, 261-265
 - topics covered, 254
 - types usable in annotations, 266-295
- functools module
 - caching properties with, 855-857
 - freezing arguments with, 247-249
 - functools.cache decorator, 320-323

- functools.lru_cache function, 323
- functools.singledispatch decorator, 326-329
- futures
 - basics of, 751-754
 - definition of term, 743
- futures.as_completed, 769-772

G

- garbage collection, 219-221, 226
- generator expressions (genexps), 26, 29, 235, 613, 818-821
- generators
 - arithmetic progression generators, 615-619
 - asynchronous generator functions, 812-818
 - examples of, 607-610
 - further reading on, 652
 - generator functions in Python standard library, 238, 619-630
 - generator-based coroutines, 777
 - generic iterable types, 639
 - humble generators, 784
 - iterable reducing functions, 630-631
 - versus iterators, 614
 - lazy generators, 610-613
 - overview of, 652
 - Sentence classes with, 606
 - significant changes to, 594
 - Soapbox discussion, 654
 - subgenerators with yield from expression, 632-639
 - topics covered, 594
 - when to use generator expressions, 613
 - yield keyword, 607
- generic classes, implementing, 541-544
- generic collections
 - parameterized generics and TypeVar, 282-286
 - Soapbox discussion, 302
 - type annotations and, 271-272
- generic functions, single dispatch, 324-329
- generic mapping types, 276
- generic static protocols, 552-554
- getattr function, 871
- getattr(object, name[, default]) function, 871
- gevent library, 711
- Global Interpreter Lock (GIL), 66, 699, 713-715, 725, 736
- globals() function, 351
- goose typing

- ABC syntax details, 457
 - ABCs in Python standard library, 449-451
 - abstract base classes (ABCs), 442-445
 - defining and using ABCs, 451-457
 - definition of term, 431
 - overview of, 446
 - structural typing with ABCs, 464-466
 - subclassing ABCs, 447-449, 458-460
 - usage of register, 463
 - virtual subclasses of ABCs, 460-463
 - gradual type system (see also type hints (type annotations))
 - abstract base classes, 278-280
 - Any type, 266-267
 - basics of, 254
 - Callable type, 291
 - further reading on, 555
 - generic collections, 271-272
 - generic mappings, 276
 - implementing generic classes, 541-544
 - implementing generic static protocols, 552-554
 - in practice, 255-260
 - Iterable, 280-282
 - legacy support and deprecated collection types, 272
 - NoReturn type, 294
 - Optional and Union types, 270
 - overloaded signatures, 520-526
 - overview of, 554
 - parameterized generics and TypeVar, 282-286
 - reading hints at runtime, 537-541
 - significant changes to, 519
 - simple types and classes, 269
 - Soapbox discussion, 557
 - static protocols, 286-291
 - subtype-of versus consistent-with relationships, 267-269
 - topics covered, 519
 - tuple types, 274-276
 - type casting, 534-537
 - TypedDict, 526
 - variance and, 544-551
 - greater than (>) operator, 577
 - greater than or equal to (>=) operator, 577
 - greenlet package, 710
- ## H
- hasattr function, 871
 - hash code, versus hash value, 84
 - hash tables, 77
 - hashable, definition of, 84
 - heapq package, 69
 - higher-order functions, 234-236
 - HTTPServer class, 503
 - HTTPX library, 782-786
 - humble generators, 784
- ## I
- I/O (input/output) (see network I/O)
 - id() function, 206
 - immutable mappings, 99
 - immutable sequences, 24
 - implicit conversion, 117
 - import time versus runtime, 306, 925
 - indefinite loops, 721
 - infix operators, 561, 574-576
 - inheritance and subclassing
 - best practices, 510-514
 - further reading on, 515
 - mixin classes, 500-502
 - multiple inheritance and method resolution order, 494-499
 - overview of, 514
 - real-world examples of, 502-509
 - significant changes to, 488
 - Soapbox discussion, 517-518
 - subclassing ABCs, 447-449, 458-460
 - subclassing built-in types, 490-494
 - super() function, 488-490
 - topics covered, 487
 - virtual subclasses of ABCs, 460-463
 - input expanding generator functions, 625
 - interfaces (see also goose typing; protocols)
 - Container interface, 15
 - further reading on, 482
 - Iterable interface, 15, 280, 435
 - overview of, 481
 - protocols as informal, 427
 - role in object-oriented programming, 431
 - significant changes to, 433
 - Sized interface, 15
 - Soapbox discussion, 484-486
 - topics covered, 433
 - typing map, 432
 - ways of defining and using, 431

- interning, 222
- invalid attribute name problem, 842
- inverted indexes, 799
- is operator, 206, 221
- .items method, 110
- iter() function, 596-599
- Iterable interface, 15, 280-282, 435-437
- iterables
 - asynchronous, 811-821
 - iterable reducing functions, 630-631
 - versus iterators, 599-603
 - unpacking, 35-38
- iterators
 - asynchronous, 811-821
 - further reading on, 652
 - versus generators, 614
 - generic iterable types, 639
 - iter() function, 596-599
 - versus iterables, 599-603
 - lazy sentences, 610-613
 - overview of, 652
 - role of, 593
 - Sentence classes with `__iter__`, 603-610
 - sequence protocol, 594-596
 - significant changes to, 594
 - Soapbox discussion, 654
 - topics covered, 594
- itertools module, 618

J

- JSON-like data, 838-842

K

- key argument, 74
- keys
 - automatic handling of missing, 90-95
 - converting nonstring keys to str, 98
 - hashability, 84
 - persistent storage for mapping, 97
 - practical consequences of using dict, 102
 - preserving key insertion order, 15, 948
 - sorting multiple, 244
- .keys method, 110
- keyword class patterns, 193
- keyword-only arguments, 242
- keywords
 - as keyword, 41
 - await keyword, 781
 - lambda keyword, 236

- nonlocal keyword, 315-317, 683
- reserved keywords, 679
- yield keyword, 238, 605-610, 614, 642, 645, 669
- KISS principle, 427, 517, 739

L

- lambda keyword, 236
- lazy sentences, 610-613
- Least Recently Used (LRU), 323
- len() function, 6
- less than (<) operator, 577
- less than or equal to (<=) operator, 577
- line breaks, 26
- lis.py interpreter
 - Environment class, 673
 - evaluate function, 676-685
 - imports and types, 671
 - OR-patterns, 686
 - parser, 671-673
 - pattern matching in, 43-47
 - Procedure class, 685-686
 - REPL (read-eval-print-loop), 675
 - Scheme syntax, 669-671
 - topics covered, 669
- list comprehensions (listcomps)
 - asynchronous, 818-821
 - building lists from cartesian products, 27
 - building sequences with, 25
 - versus generator expressions, 29
 - local scope within, 26
 - readability and, 25
 - syntax tip, 26
 - versus map and filter functions, 27, 235
- lists
 - alternatives to, 59-70
 - building lists of lists, 51
 - list.sort versus sorted built-in, 56-58
 - mixed-bag, 74
 - multiline, 26
 - shallow copies of, 208-211
 - versus tuples, 34
 - using tuples as immutable, 32-34
- locks, definition of term, 699
- LRU (see Least Recently Used)

M

- magic methods, 4, 19
- managed attributes, 882

- managed classes, 881
- managed instances, 882
- map function, 27, 234-236
- mappings
 - automatic handling of missing keys, 90-95
 - case-insensitive, 500-502
 - immutable mappings, 99
 - mapping generator functions, 620
 - merging, 80
 - pattern matching with, 81-83
 - standard API of mapping types, 83-90
 - unpacking, 80
- match blocks (see with, match, and else blocks)
- match/case statement, 38, 81
- mathematical vector operations, 566
- max() function, 521-526
- memoization, 320-323
- memory, saving with `__slots__`, 384-388
- memoryview class, 62-64
- metaclasses
 - basics of, 931-933
 - considerations for use, 947-949
 - customizing classes, 933
 - definition of term, 910
 - example metaclass, 934-937
 - metaclass evaluation time experiment, 937-942
 - metaclass solution for checkedlib.py, 942-947
 - useful applications of, 952
- metaobjects, 19
- metaprogramming, **xxii** (see also attribute descriptors; class metaprogramming; dynamic attributes and properties)
- method resolution order (MRO), 494-499
- methods, as callable objects, 238 (see also sequences, special methods for; special methods)
- Meyer's Uniform Access Principle, 875-876
- Mills & Gizmos Notation (MGN), 883
- mixin classes, 500-502
- mixin methods, 502
- modulo (%) operator, 5, 12
- monkey-patching, 438-440, 486, 897
- MRO (see method resolution order)
- multicore processing
 - data science, 727
 - distributed task queues, 732
 - increased availability of, 725
 - server-side web/mobile development, 728
 - system administration, 726
 - WSGI application servers, 730
- multiline lists, 26
- multiple inheritance (see also inheritance and subclassing)
 - method resolution order and, 494-499
 - real-world examples of, 502-509
- multiplication, scalar, 10, 572-574
- multiprocessing package, 69, 704
- mutable objects, 225 (see also object references)
- mutable parameters, 214-218
- mutable sequences, 24
- mutable values, inserting or updating, 87-90
- MutableMapping ABC, 83
- Mypy type checker, 255-260
- `my_fmt.format()` method, 5

N

- name mangling, 382
- namedtuple, 169-172
- native coroutines
 - versus asynchronous generators, 818
 - definition of term, 777
 - functions defined with `async def`, 238
 - humble generators and, 784
- network I/O
 - downloading with `asyncio`, 782-786
 - downloading with `concurrent.futures`, 749-750
 - downloads with progress display and error handling, 762-772
 - enhancing `asyncio` downloader, 787-797
 - essential role of concurrency in, 744-746
 - myth of I/O-bound systems, 826
 - role of futures, 751-754
 - sequential download script, 746-749
- NFC (see Normalization Form C)
- nominal typing, 262
- nonlocal keyword, 315-317, 683
- nonoverriding descriptors, 855, 892-898
- NoReturn type, 294
- Normalization Form C (NFC), 140
- normalized text matching, 143-148
- not equal to (!=) operator, 577
- numbers ABCs, 478-481
- numbers package, 279
- numeric protocols, 478-481
- numeric tower, 279

- numeric types
 - checking for convertibility, 468, 480
 - emulating using special methods, 9-12
 - hashability of, 84
 - support for, 479

- NumPy, 64-67

O

- object references
 - aliasing, 204-208
 - deep copies, 211-213
 - del and garbage collection, 219-221
 - distinction between objects and their names, 201
 - function parameters as references, 213-218
 - further reading on, 224
 - immutability and, 221
 - overview of, 223
 - shallow copies, 208-211
 - Soapbox discussion, 225
 - variables as labels versus boxes, 202-204

- objects
 - callable objects, 237-239, 598-599
 - first-class, 231
 - flexible object creation, 843-845
 - mutable, 225
 - treating functions like, 232-234
 - user-defined callable objects, 239

- operator module, 243-247

- operator overloading
 - augmented assignment operators, 580-585
 - basics of, 562
 - further reading on, 587
 - infix operator method names, 576
 - infix operators, 561
 - overloading * for scalar multiplication, 572-574
 - overloading + for vector addition, 566-572
 - overview of, 585
 - rich comparison operators, 577-580
 - significant changes to, 562
 - Soapbox discussion, 588
 - topics covered, 562
 - unary operators, 563-566
 - using @ as infix operator, 574-576

- Optional type, 270

- OR-patterns, 686

- OrderedDict, 85, 95

- os functions, str versus bytes in, 156

- overloaded signatures, 520-526

- overriding descriptors, 855, 863, 892-898

P

- parallelism, 695, 697

- parameterized decorators, 329-335

- parameterized types, 544

- parameters
 - annotating positional only and variadic parameters, 295
 - introspection of function parameters, 232
 - keyword-only, 242
 - mutable, 214-218
 - parameter passing, 213
 - positional, 240-243

- pattern matching
 - *_ symbol, 42
 - destructuring, 40
 - in lis.py interpreter, 43-47, 669-687
 - with mappings, 81-83
 - match/case statement, 38
 - pattern matching class instances, 192-195
 - tuples and lists, 41
 - type information, 42
 - _ symbol, 41

- patterns (see functions, design patterns with first-class; pattern matching)

- pickle module, 97

- Pingo library, 99

- pipe (|) operator, 80, 686

- pipe equals (|=) operator, 80

- plain text, 129, 161

- .pop method, 67

- positional class patterns, 194

- positional parameters, 240-243

- positional patterns, 377

- process pools
 - code for multicore prime checker, 719-723
 - example problem, 716
 - process-based solution, 718
 - thread-based nonsolution, 724
 - understanding elapsed times, 718
 - varying process numbers, 723

- processes
 - definition of term, 698
 - launching with concurrent.futures, 754-758

- progress displays, 762-772

- Project Jupyter, 727

- proper tail calls (PTC), 691-693

- properties (see computed properties; dynamic attributes and properties)
- property class, 860-865
- protocol classes, 403
- Protocol type, 286-291
- protocols (see also interfaces)
 - defensive programming, 440-442
 - duck typing and, 402
 - further reading on, 482
 - implementing at runtime, 438-440
 - implementing generic static protocols, 552-554
 - as informal interfaces, 427
 - meanings of protocol, 434
 - numeric, 478-481
 - overview of, 481
 - sequence and iterable protocols, 435-437
 - significant changes to, 433
 - Soapbox discussion, 484-486
 - static protocols, 403, 466-481
 - topics covered, 433
- PSF (see Python Software Foundation)
- PUG (see Python Users Group)
- PyICU, 150
- PyLadies, 960
- pytest package, xxii
- Python
 - appreciating language-specific features, xix
 - approach to learning, xx-xxii
 - community support for, 959
 - fluentpython.com, xxiii
 - functional programming with, 250
 - functioning with multicore processors, 725-733
 - further reading on, 960
 - prerequisites to learning, xx
 - target audience, xx
 - versions featured, xx
- Python Data Model
 - further reading on, 18
 - __getitem__ and __len__, 5-8
 - making len work with custom objects, 17
 - overview of, 3, 18
 - significant changes to, 4
 - Soapbox discussion, 19
 - special methods overview, 15-17
 - using special methods, 8-15
- Python Software Foundation (PSF), 960
- Python type checkers, 255

- Python Users Group (PUG), 960
- python-tulip list, 959
- Pythonic Card Deck example, 5-8
- Pythonic objects (see also objects)
 - alternative constructor for, 368
 - building user-defined classes, 363
 - classmethod versus staticmethod, 369
 - formatted displays, 370-374
 - further reading on, 392
 - hashable Vector2d, 374-377
 - object representations, 364
 - overriding class attributes, 389-391
 - overview of, 391
 - private and protected attributes, 382-384
 - saving memory with __slots__, 384-388
 - significant changes to, 364
 - Soapbox discussion, 394-396
 - supporting positional patterns, 377
 - topics covered, 363
 - Vector2d class example, 365-368
 - Vector2d full listing, 378-381
- Pythonic sums, 428-430
- pyuca library, 150, 158

Q

- quantity properties, 865-868
- questions and comments, xxv
- queues
 - definition of term, 698
 - deque (double-ended queue), 59, 67
 - distributed task queues, 732
 - implementing, 69

R

- race conditions, 723
- random.choice function, 6
- recycling (see garbage collection)
- reduce function, 234-236
- reducing functions, 412, 630-631
- refactoring strategies
 - choosing the best, 350
 - classic, 342-346
 - Command pattern, 355-357
 - decorator-enhanced pattern, 353-355
 - finding strategies in modules, 351-353
 - function-oriented, 347-350
- reference counting, 219
- registration decorators, 308, 329-332
- regular expressions, str versus bytes in, 155

- repr() function, 364
- reserved keywords, 679
- rich comparison operators, 577-580
- running averages, computing, 643-645

S

- S-expression, 669
- salts, 85
- Scheme language, 43-47, 669-671
- SciPy, 64-67
- scope
 - dynamic scope versus lexical scope, 338-339
 - function local scope, 310
 - module global scope, 310
 - variable scope rules, 308-310
 - within comprehensions and generator expressions, 26
- semaphores, 790-794
- Sentence classes, 603-610
- sentinels, 721
- sequence protocol, 435-437, 594-596
- sequences
 - alternatives to lists, 59-70
 - further reading on, 71
 - list comprehensions and generator expressions, 25-30
 - list.sort versus sorted built-in, 56-58
 - overview of, 70
 - overview of built-in, 22-24
 - pattern matching with, 38-47
 - significant changes to, 22
 - slicing, 47-50
 - Soapbox discussion, 73-75
 - topics covered, 22
 - tuples, 30-35
 - uniform handling of, 21
 - unpacking sequences and iterables, 35-38
 - using + and * with, 50-56
- sequences, special methods for
 - applications beyond three dimensions, 398
 - dynamic attribute access, 407-411
 - __format__, 418-425
 - further reading on, 426
 - __hash__ and __eq__, 411-416
 - overview of, 425
 - protocols and duck typing, 402
 - significant changes to, 398
 - sliceable sequences, 403-407
 - Soapbox discussion, 427-430

- topics covered, 397
- Vector implementation strategy, 398
- Vector2d compatibility, 399-401
- sequential.py program, 716
- server-side web/mobile development, 728
- servers
 - Asynchronous Server Gateway Interface (ASGI), 732
 - HTTPServer class, 503
 - TCP servers, 804-811
 - test servers, 765
 - ThreadingHTTPServer class, 503
 - Web Server Gateway Interface (WSGI), 730
 - writing asyncio servers, 799-811
- setattr function, 871
- sets (see also dictionaries and sets)
 - consequences of how set works, 107
 - set comprehensions, 106
 - set literals, 105
 - set operations, 107-110
 - set operations on dict views, 110
 - set theory, 103-105
- shallow copies, 208-211
- shelve module, 97
- simple class patterns, 192
- single dispatch generic functions, 324-329
- Sized interface, 15
- slicing
 - assigning to slices, 50
 - excluding last item in, 47
 - multidimensional slicing and ellipses, 49
 - slice objects, 48
 - sliceable sequences, 403-407
- Soapbox sidebars
 - @dataclass, 197
 - anonymous functions, 251
 - __call__, 360
 - code points, 161
 - data model versus object model, 19
 - design patterns, 359
 - duck typing, 302, 427
 - dynamic scope versus lexical scope, 338-339
 - equality (==) operator, 225
 - explicit self argument, 905
 - flat versus container sequences, 73
 - __format__, 428
 - function-class duality, 876
 - functional programming with Python, 250
 - generic collections, 302

- inheritance across languages, 518
- interfaces, 486
- key argument, 74
- lis.py and evaluate function, 693
- magic methods, 19
- metaobjects, 19
- minimalistic iterator interface, 654
- mixed-bag lists, 74
- monkey-patching, 486
- multilevel class hierarchies, 517
- mutability, 225
- non-ASCII names in source code, 160
- object destruction and garbage collection, 226
- operator overloading, 588
- Oracle, Google, and the Timbot, 74
- plain text, 161
- pluggable generators, 654-656
- programming language design, 956
- proper tail calls (PTC), 691-693
- properties and up-front costs, 394
- protocols as informal interfaces, 427
- Python decorators and decorator design pattern, 339
- Pythonic sums, 428-430
- safety versus security in private attributes, 395
- static typing, 484
- syntactic sugar, 114
- thread avoidance, 773
- threads-and-locks versus actor-style programming, 739-741
- trade-offs of built-ins, 517
- tuples, 73
- Twisted library, 830
- type hints (type annotations), 299-302
- typing map, 485
- undocumented classes, 557
- Uniform Access Principle, 875-876
- uvloop, 829
- variance notation in other classes, 558
- with statements, 691
- sorted function, 56-58
- special methods (see also sequences, special methods for)
 - advantages of using, 6
 - Boolean values of custom types, 13
 - calling, 8
 - Collection API, 14-15
 - emulating numeric types, 9-12
 - `__getitem__` and `__len__`, 5-8
 - naming conventions, 3
 - purpose of, 3
 - special method names (operators excluded), 15
 - special method names and symbols for operators, 16
 - string representation, 12
- spinners (loading indicators) (see also network I/O)
 - comparing supervisor functions, 711
 - created using coroutines, 706-710
 - created with multiprocessing package, 704
 - created with threading, 701-704
 - Global Interpreter Lock impact, 713-715
 - keeping alive, 715
- square brackets (`[]`), 6, 26, 35, 49
- stacked decorators, 322
- star (*) operator, 10, 36-37, 50-56, 240, 572-574
- star equals (=) operator, 580-585
- static duck typing, 291, 432, 515
- static protocols
 - best practices for protocol design, 476
 - definition of, 435
 - designing, 474-476
 - versus dynamic protocols, 403
 - extending, 477
 - implementing generic static protocols, 552-554
 - limitations of runtime protocol checks, 471
 - numbers ABCS and numeric protocols, 478-481
 - runtime checkable, 468-471
 - Soapbox discussion, 484
 - supporting, 472-474
 - type hints (type annotations), 286-291
 - typed double function, 466-468
- static typing, 431
- staticmethod decorator, 369
- storage attributes, 882
- `str()` function, 13, 364
- `str.format()` method, 5, 370
- Strategy pattern, 342-346
- strings
 - default sorting of, 58
 - dual-mode str and bytes APIs, 155-157
 - normalizing Unicode for reliable comparisons, 140-148

- representation using special methods, 12
- strong testing, 296
- struct module, 118
- structural typing, 464-466
- structured concurrency, 823
- subclassing (see inheritance and subclassing)
- subgenerators, 632
- subtype-of relationships, 267
- super() function, 411, 488-490
- syntactic sugar, 114
- SyntaxError, 128
- system administration, 726

T

- tail call optimization (TCO), 691-693
- TCP servers, 804-811
- TensorFlow, 727
- test servers, 765
- text files, handling, 131-139 (see also Unicode text versus bytes)
- ThreadingHTTPServer class, 503
- ThreadingMixIn class, 503
- threads
 - definition of term, 698
 - enhancing asyncio downloader, 788-797
 - further reading on, 734
 - Global Interpreter Lock impact, 714
 - spinners (loading indicators) using, 701-704
 - thread avoidance, 773
 - thread-based process pools, 724
- throttling, 790-794
- Timsort algorithm, 74
- Tkinter GUI toolkit
 - benefits and drawbacks of, 513
 - multiple inheritance in, 507-509
- tree structures, traversing, 634-639
- tuples
 - classic named tuples, 169-172
 - immutability and, 221
 - as immutable lists, 32-34
 - versus lists, 34
 - nature of, 73
 - as records, 30-32
 - relative immutability of, 207
 - simplified memory diagram for, 23
 - tuple unpacking, 32
 - type hints (type annotations), 274-276
 - typing.NamedTuple, 172
- Twisted library, 830

- type casting, 534-537
- type hints (type annotations)
 - annotating positional only and variadic parameters, 295
 - for asynchronous objects, 824
 - basics of, 173-179
 - benefits and drawbacks of, 253
 - flawed typing and strong testing, 296
 - further reading on, 298
 - generic type hints for coroutines, 650
 - gradual typing, 254-260 (see also gradual type system)
 - overview of, 297
 - significant changes to, 254
 - Soapbox discussion, 299-302
 - supported operations and, 261-265
 - topics covered, 254
 - types usable in, 266-295
- typed double function, 466-468
- TypedDict, 164, 526-534
- Typeshed project, 280
- TypeVar, 282-286
- typing map, 432, 485 (see also type hints (type annotations))
- typing module, 266
- typing.NamedTuple, 172

U

- UCA (see Unicode Collation Algorithm)
- UCS-2 encoding, 124
- UML class diagrams
 - ABCs in collections.abc, 449
 - annotated with MGN, 882, 943
 - Command design pattern, 355
 - django.views.generic.base module, 504
 - django.views.generic.list module, 506
 - fundamental collection types, 14
 - managed and descriptor classes, 880
 - MutableSequence ABC and superclasses, 448
 - Sequence ABC and abstract classes, 436
 - simplified for collections.abc, 24
 - simplified for MutableMapping and superclasses, 83
 - simplified for MutableSet and superclasses, 107
 - Strategy design pattern, 342
 - Tkinter Text widget class and superclasses, 498

- TomboList, 461
- unary operators, 563-566
- undocumented classes, 557
- Unicode Collation Algorithm (UCA), 150
- Unicode literals escape notation (`\N{}`), 136
- Unicode sandwich, 131
- Unicode text versus bytes
 - basic encoders/decoders, 123-124
 - byte essentials, 120-123
 - characters and Unicode standard, 118-120
 - dual-mode str and bytes APIs, 155-157
 - further reading on, 158
 - handling text files, 131-139
 - normalizing Unicode for reliable comparisons, 140-148
 - overview of, 157
 - significant changes to, 118
 - Soapbox discussion, 160
 - sorting Unicode text, 148-150
 - topics covered, 117
 - understanding encode/decode problems, 125-131
- Unicode database, 150-155
- UnicodeDecodeError, 126
- UnicodeEncodeError, 125
- Uniform Access Principle, 875-876
- Union type, 270
- unittest module, xxii
- unpacking
 - iterables and mappings, 240
 - mapping unpackings, 80
 - nested, 37
 - sequences and iterables, 35
 - using `*` to grab excess items, 36
 - with `*` in function calls and sequence literals, 37
- user-defined functions, 238
- UserDict, 97-99
- UTF-8 decoding, 129
- UTF-8-SIG encoding, 130
- uvloop, 829

V

- variable annotations
 - meaning of, 175-179
 - syntax of, 174
- variable scope rules, 308-310
- variables
 - free, 313

- init-only variables, 186
 - as labels versus boxes, 202-204
 - lookup logic, 316
- variadic parameters, 295
- variance
 - contravariant types, 547
 - covariant types, 546
 - in callable types, 292
 - invariant types, 545
 - overview of, 549
 - relevance of, 544
 - rules of thumb, 551
 - variance notation in other classes, 558
- vars([object]) function, 871
- Vector class, multidimensional
 - applications beyond three dimensions, 398
 - dynamic attribute access, 407-411
 - `__format__`, 418-425
 - further reading on, 426
 - `__hash__` and `__eq__`, 411-416
 - implementation strategy, 398
 - overview of, 425
 - protocols and duck typing, 402
 - sliceable sequences, 403-407
 - topics covered, 397
 - Vector2d compatibility, 399-401
- Vector2d
 - class example, 365-368
 - full listing, 378-381
 - hashable, 374
- vectors
 - overloading + for vector addition, 566-572
 - representing two-dimensional, 9-12
- virtual attributes, 835
- virtual subclasses, 460-463

W

- Walrus operator (`:=`), 26
- weak references, 221
- Web Server Gateway Interface (WSGI), 730
- web/mobile development, 728
- with, match, and else blocks
 - context managers and with blocks, 658-669
 - else clause, 687-689
 - further reading on, 690
 - overview of, 689
 - pattern matching in `lis.py`, 669-687
 - purpose of with statements, 658
 - significant changes to, 658

Soapbox discussion, 691-693
topics covered, 657

Y

yield from expression, 632-639

yield keyword, 238, 605-610, 614, 642, 645, 669

Z

zero-based indexing, 47

zip() function, 416

About the Author

Luciano Ramalho was a web developer before the Netscape IPO in 1995, and switched from Perl to Java to Python in 1998. He joined Thoughtworks in 2015, where he is a Principal Consultant in the São Paulo office. He has delivered keynotes, talks, and tutorials at Python events in the Americas, Europe, and Asia, and also presented at Go and Elixir conferences, focusing on language design topics. Ramalho is a fellow of the Python Software Foundation and cofounder of Garoa Hacker Clube, the first hackerspace in Brazil.

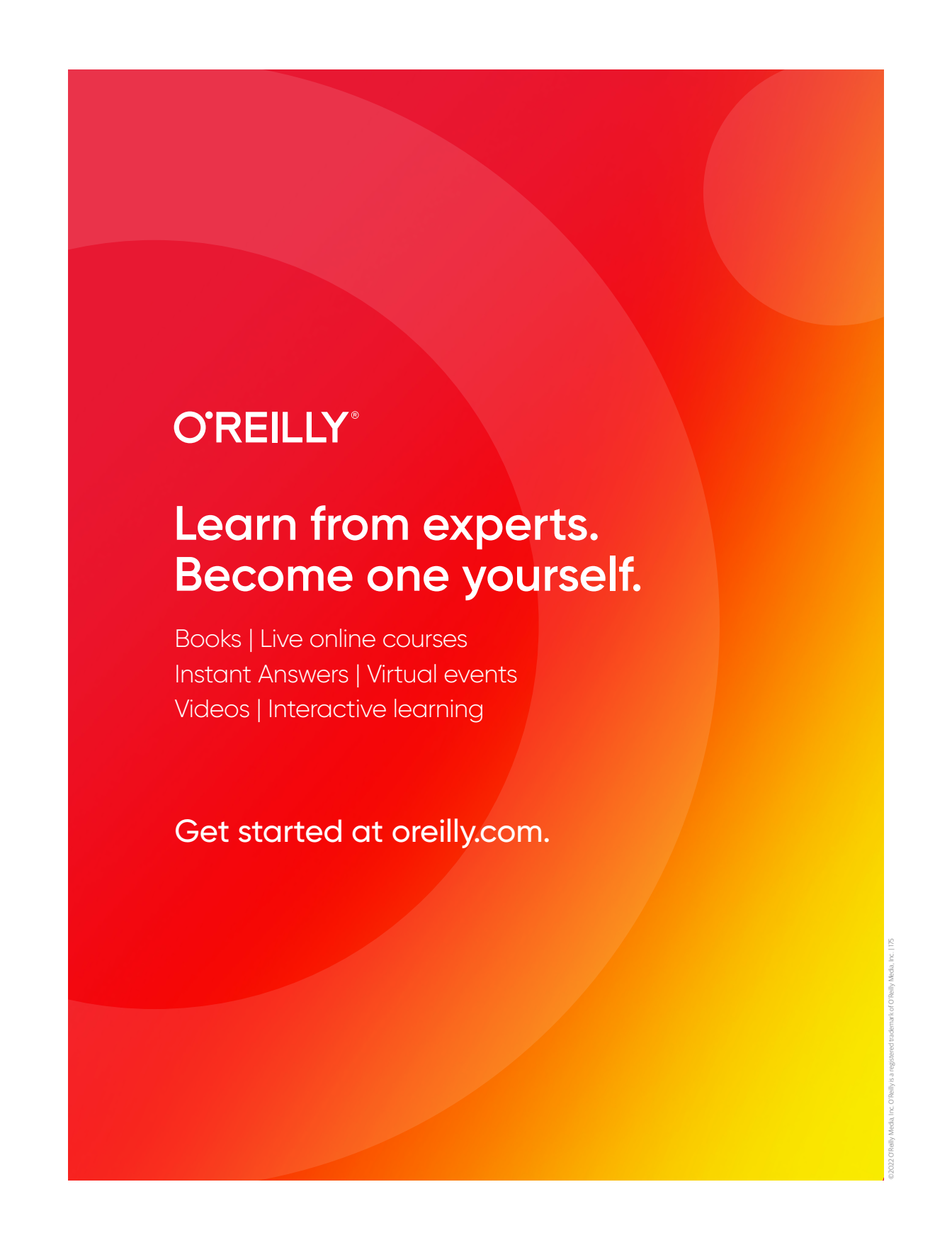
Colophon

The animal on the cover of *Fluent Python* is a Namaqua sand lizard (*Pedioplanis namaquensis*), found throughout Namibia in arid savannah and semi-desert regions.

The Namaqua sand lizard has a black body with four white stripes running down its back, brown legs with white spots, a white belly, and a long, pinkish-brown tail. It is one of the fastest of the lizards active during the day and feeds on small insects. It inhabits sparsely vegetated sand gravel flats. Female Namaqua sand lizards lay between three to five eggs in November, and these lizards spends the rest of winter dormant in burrows that they dig near the base of bushes.

The current conservation status of the Namaqua sand lizard is of “Least Concern.” Many of the animals on O’Reilly covers are endangered; all of them are important to the world.

The cover illustration is by Karen Montgomery, based on a black and white engraving from Wood’s *Natural History*. The cover fonts are Gilroy Semibold and Guardian Sans. The text font is Adobe Minion Pro; the heading font is Adobe Myriad Condensed; and the code font is Dalton Maag’s Ubuntu Mono.

The background of the entire page is a vibrant gradient of red and orange. Overlaid on this are several large, semi-transparent circles in various shades of red and orange, creating a layered, organic effect.

O'REILLY®

**Learn from experts.
Become one yourself.**

Books | Live online courses
Instant Answers | Virtual events
Videos | Interactive learning

Get started at oreilly.com.